

გაკვეთილი 1 Mod 4 (4.1)

გაკვეთილი 1 — TypeScript-ის შესავალი და ინსტალაცია

გაკვეთილის მიზანი

გაკვეთილის ბოლოს მოსწავლემ უნდა იცოდეს:

- რა არის TypeScript;
- რა განსხვავებაა JavaScript-სა და TypeScript-ს შორის;
- რატომ ვიყენებთ TypeScript-ს;
- როგორ დავაინსტალიროთ TypeScript;
- როგორ შევქმნათ .ts ფაილი;
- როგორ გადავაქციოთ TypeScript კოდი JavaScript-ად;
- როგორ გავუშვათ პირველი პროგრამა.

1. რა არის TypeScript?

აქამდე JavaScript-ში ვწერდით მაგალითად:

```
let name = "Giorgi";  
let age = 20;  
  
console.log(name);  
console.log(age);
```

JavaScript-ში ცვლადის შექმნისას არ ვუთითებთ, რა ტიპის მონაცემი უნდა შევინახოთ მასში.

მაგალითად ეს შესაძლებელია:

```
let age = 20;  
  
age = "twenty";
```

თავიდან `age` იყო რიცხვი, შემდეგ კი ტექსტი გახდა.

დიდ პროექტებში ამან შეიძლება შეცდომები გამოიწვიოს.

TypeScript არის **JavaScript**-ის გაფართოებული ვერსია, რომელიც საშუალებას გვაძლევს წინასწარ მივუთითოთ მონაცემის ტიპი.

მაგალითად:

```
let age: number = 20;
```

აქ ვამბობთ:

`age` ცვლადში მხოლოდ რიცხვი უნდა იყოს.

ამიტომ თუ დავწერთ:

```
age = "twenty";
```

TypeScript შეცდომას გვაჩვენებს.

2. JavaScript და TypeScript

ძალიან მარტივად შეგიძლია ბავშვებს ასე აუხსნა:

JavaScript:

```
let username = "Nika";  
  
username = 25;
```

JavaScript ამას დაუშვებს.

TypeScript:

```
let username: string = "Nika";  
  
username = 25;
```

TypeScript გვეტყვის, რომ შეცდომაა.

რატომ?

რადგან ჩვენ დავწერეთ:

```
: string
```

რაც ნიშნავს, რომ `username` უნდა იყოს ტექსტური მნიშვნელობა.

3. რატომ გვჭირდება TypeScript?

წარმოიდგინეთ, გვაქვს პროგრამა, სადაც მომხმარებლის ასაკს ვინახავთ.

```
let age: number = 18;
```

შემდეგ პროგრამაში გვინდა:

```
console.log(age + 5);
```

შედეგი:

```
23
```

მაგრამ თუ `age` შემთხვევით ტექსტი გახდებოდა:

```
let age = "18";
```

```
console.log(age + 5);
```

მივიღებდით:

```
185
```

სწორედ ასეთი შეცდომების თავიდან აცილებაში გვეხმარება TypeScript.

TypeScript-ის მთავარი უპირატესობები

1. შეცდომებს ხშირად კოდის გაშვებამდე გვაჩვენებს;
2. ვიცით, რა ტიპის მონაცემს შეიცავს ცვლადი;
3. დიდი პროექტის კოდი უფრო მარტივი გასაგებია;
4. პროგრამის შეცდომების პოვნა უფრო მარტივია.

4. TypeScript-ის ინსტალაცია

ჯერ საჭიროა კომპიუტერში **Node.js** და **npm**.

ტერმინალში შეგვიძლია შევამოწმოთ:

```
node -v
```

შემდეგ:

```
npm -v
```

თუ ვერსიები გამოჩნდა, შეგვიძლია TypeScript-ის დაყენება.

```
npm install -g typescript
```

ინსტალაციის შემდეგ ვამოწმებთ:

```
tsc -v
```

tsc ნიშნავს **TypeScript Compiler**-ს.

5. პირველი TypeScript ფაილი

JavaScript ფაილის გაფართოებაა:

```
.js
```

მაგალითად:

```
script.js
```

TypeScript ფაილის გაფართოებაა:

```
.ts
```

მაგალითად შევქმნათ:

```
script.ts
```

და ჩაწეროთ:

```
let studentName: string = "Nika";  
let age: number = 17;  
  
console.log(studentName);  
console.log(age);
```

6. TypeScript პირდაპირ ბრაუზერში არ სრულდება

ეს მნიშვნელოვანი ნაწილია.

ბრაუზერს ჩვეულებრივ JavaScript სჭირდება. ამიტომ TypeScript კოდი ჯერ უნდა გადავაქციოთ JavaScript-ად.

ამ პროცესს ეწოდება **Compilation** — კომპილაცია.

გვაქვს:

```
script.ts
```

↓ კომპილაცია

```
script.js
```

ამისთვის ტერმინალში ვწერთ:

```
tsc script.ts
```

ამის შემდეგ შეიქმნება:

```
script.js
```

7. პირველი მაგალითი

script.ts :

```
let firstName: string = "Ana";  
let age: number = 16;  
  
console.log("Name:", firstName);  
console.log("Age:", age);
```

ვაკომპილირებთ:

```
tsc script.ts
```

შემდეგ შეგვიძლია გავუშვათ:

```
node script.js
```

შედეგი:

```
Name: Ana  
Age: 16
```

8. პირველი ტიპები

ამ გაკვეთილზე მხოლოდ სამი ძირითადი ტიპი გააცანი.

string — ტექსტი

```
let city: string = "Kutaisi";
```

number — რიცხვი

```
let age: number = 17;
```

ასევე:

```
let price: number = 25.5;
```

boolean — true ან false

```
let isStudent: boolean = true;
```

ან:

```
let isAdmin: boolean = false;
```

ერთად:

```
let username: string = "Giorgi";  
let age: number = 20;  
let isStudent: boolean = true;  
  
console.log(username);  
console.log(age);  
console.log(isStudent);
```

9. შეცდომის მაგალითი

დაწერე:

```
let score: number = 50;  
  
score = "fifty";
```

ჰკითხე ბავშვებს:

რატომ არის შეცდომა?

იმიტომ, რომ:

```
score: number
```

ნიშნავს, რომ `score` -ში რიცხვი უნდა შევიდეს, `"fifty"` კი `string` -ია.

სწორი იქნება:

```
score = 70;
```

პრაქტიკული ნაწილი — გაკვეთილი 1

სავარჯიშო 1

შექმენით სამი ცვლადი:

- თქვენი სახელი;
- ასაკი;
- ხართ თუ არა მოსწავლე.

სწორი ტიპების გამოყენებით.

მაგალითად:

```
let name: string = "Nika";  
let age: number = 16;  
let isStudent: boolean = true;  
  
console.log(name);  
console.log(age);  
console.log(isStudent);
```

სავარჯიშო 2

იპოვეთ შეცდომები:

```
let city: string = 25;  
let age: number = "17";  
let isStudent: boolean = "true";
```

სწორი ვერსია:

```
let city: string = "Khoni";  
let age: number = 17;  
let isStudent: boolean = true;
```

სავარჯიშო 3

შექმენით:

```
student.ts
```

მასში მოსწავლემ შეინახოს:

- სახელი;
- გვარი;
- ასაკი;
- ქულა;
- არის თუ არა გამოცდა ჩაბარებული.

შემდეგ ყველაფერი გამოიტანოს `console.log()` -ით.